



NUS
National University
of Singapore

CDE2310: FUNDAMENTALS OF SYSTEMS DESIGN

GROUP 2 G1 REPORT

<u>Team Members:</u>	
REAGAN MOSES WIDJAJA	A0307076E
LOO ZHONG EN SAMUEL	A0322527J
UDAIYAN BHAN	A0336759L
ARAVIND THEJUS	A0332970A

Introduction (Step 1: Problem Definition)

1.1 Mission Overview

For this project, we are tasked to design and build an Autonomous Mobile Robot that can handle and execute complex warehouse logistics. The robot must be able to self-navigate, identify visual landmarks, and deliver its payloads precisely, and sequentially if required, at each station. As a challenge, one of the stations will have a moving target.

1.2 Mission Objectives

Our mission objectives can be broken down into two stages:

Stage 1: Primary Objectives

1. Identify the Station through Visual Markers
2. Align and dock within allowable docking distance
3. Track the motion profile of the target, if required
4. Unload a payload batch (3 Ping Pong Balls) into the target receptacle, in fixed timing sequence if required

Stage 2: Bonus Objectives

1. Identify the Lift Lobby & Final station through Visual Markers
2. Initiate and Use API Calls to control a lift
3. Safely travel to the second level
4. Unload the final payload batch (3 Ping Pong Balls) at the final station

1.3 Constraints

The mission has the following constraints:

1. Timing
 - a. Mission set-up, deployment and teardown must be done within 25 minutes
 - b. The Robot System must be designed, built and fully functional by Week 12
2. Navigation
 - a. The Robot must rely on its sensors' data to map out its surroundings and for navigation.
 - b. Navigation methods which uses line-following is not allowed
3. Environment
 - a. The gaps between maze wall panels may cause issues the LiDAR reading

1.4 System Requirements

For our robot to successfully complete the mission, we have set the following requirements:

Functional Requirements

1. The Robot must navigate to the delivery zones autonomously
2. The Robot must be able to identify and read visual markers
3. The Robot must be able to unload the payload batch at each station, following a given timing sequence if required
4. (Bonus) The Robot must be able to initiate and emit API Calls

Performance Requirements

1. The whole Mission must be completed within 25 minutes
2. The Robot must not collide with any walls
3. The Payload delivered must stay inside the target receptacle
4. The Payload Delivery system must have a success rate of at least 95%
5. The Robot must be able to track the target's motion profile with at least 90% accuracy
6. The Robot must map out the maze entirely

Subsystem Research (Step 2: Literature Review)

Given the limited time we have to design and build our robot, and the hardware limitations of the Turtlebot 3 platform, we have limited our scope of research to solutions that have already been implemented or have shown to work well with the Turtlebot 3 system. We have broken up our research into 3 key sub-systems: **Navigation, Visual Marker Identification & Motion Tracking, and Ball Launcher Mechanism.**

2.1 Navigation System

Currently, the Turtlebot 3 we have on hand has some hardware and software features available, as part of its Robot Operating System (ROS), that we can make use of to perform Simultaneous Localisation and Mapping (SLAM), namely,

1. RViz
 - a. Visualisation Software to see where the Robot is and its surroundings
2. Cartographer
 - a. Provides an Occupancy Map of the robot's surroundings by indicating free, occupied and unknown (potentially unexplored) spaces
3. LiDAR
 - a. Provides distance measurements of obstacles all around (360°) the robot
4. Inertial Measurement Unit (IMU)
 - a. Provides Odometry Data which gives us the Robot's Pose and Orientation

5. Nav2 Stack

- a. A powerful navigation package which provides controller plugins for navigation and path planning and execution

Given the nature of the mission and the complexity in integrating it with our other subsystems, we will not make use of the Nav2 stack in this case. We will, however, make use of data from the LiDAR, IMU and the OccupancyGrid topic to aid in our visualisation, mapping and navigation. We can split the navigation stage into two stages:

Stage 1: Path Planning

In this stage, we will identify a viable path that the robot can follow which will be in the form of waypoints. This can be done through various algorithms, namely:

1. Wall Follower
 - a. In this algorithm, the robot will detect the walls on either its left or right side and follow it at a safe distance away. Assuming that the maze walls are connected and there are no loops, the robot will be able to explore and map out the entire maze.
2. Frontiers Exploration
 - a. Frontiers are defined as the boundary between explored areas and unexplored areas in a given map.
 - b. In this algorithm, the robot will make use of the Occupancy Data of its current surroundings to identify the boundaries between the known free spaces and unknown areas of the map, while taking into account the obstacles present. To do this, we can make use of a few search algorithms:
 - i. Breadth First Search (BFS)
 - Taking the robot's position on the map as the start point, the algorithm searches points closest to the robot first and gradually expands out to find Frontiers.
 - This Algorithm finds the frontier that is closest to the Robot and returns the shortest path to it.
 - ii. Depth First Search (DFS)
 - Taking the robot's position on the map as the start point, the algorithm, instead of searching adjacent points on the map and expanding outwards, DFS dives deep into the map to locate frontiers
 - This algorithm finds Frontiers that are further away from the robot.
 - c. Once a Frontier has been identified, we can set this point as a "Goal" point. By exhausting all frontiers, the robot will be able to map the entire maze.
3. Best Search First (A*)
 - a. For this algorithm to work, a 'goal point' must be identified first. This could be a form of a 'frontier' or a set point in the maze.
 - b. Once the Goal Point has been set, the algorithm plans a path to the goal by taking into account the relative 'cost' of each waypoint. This 'cost' could include the distance from the robot to the waypoint, the distance from the waypoint to the goal and potential obstacles nearby.

Stage 2: Path Execution

In this stage, we will execute the path that has been identified. We can make use of the following algorithms:

1. Regulated Pure Pursuit (RPP)
 - a. Similar to regular pure pursuit algorithms, RPP calculates the curvature of the path required to reach the waypoint that is at or near to "Lookahead Point" (Lookahead Point is defined as a target point that is in front of the robot).
 - b. RPP uses the curvature of the path to calculate to adjust both the speed of the robot and the lookahead point, hence giving the robot better stability and control while negotiating turns.
2. Dynamic Window Approach (DWA)
 - a. DWA makes use of the robot's dynamics (speed, angular velocity) and creates a 'map' of potential velocities that the robot might reach as it travels. As a result, DWA ensures that the robot only operates within its physical limits, especially when encountering obstacles.

2.2 Visual Marker Identification & Motion Tracking

Currently, the Turtlebot 3 is equipped with the Raspberry Pi Camera V2 to provide us with the necessary camera feed. For our Visual Marker, we can make use of the following options:

1. QR Codes
 - a. QR Codes can encode large amounts of data. This is useful as in our mission, QR Codes can be used to store information about the station
 - b. QR Codes only offer basic levels of motion tracking as it has a limited viewing angle as well as readability distance
2. ArUco Markers
 - a. ArUco markers are widely used in Augmented Reality (AR) Applications as it provides good pose estimation data. This is particularly useful in our mission context as ArUco Markers can be used to help align our robot with the docking station and track the dynamic target in station B.

- b. ArUco markers can only store its Identification Number (ID)

We also need to consider the image sensor to provide the video stream. For that, we can consider the following options:

1. Raspberry Pi Camera Module 2
 - a. Direct from Raspberry Pi, this camera is equipped with an 8MP sensor that is capable of 1080p or 480p Video Streaming at 45 or 100 Frames Per Second (FPS) respectively. It connects to the Raspberry Pi 4B via onboard CSI Connector
2. Logitech CS270 USB Camera
 - a. Popular Budget Friendly USB Camera that supports 720p Video streaming at 30 FPS and has a Field of View at 55° and resolution of 0.9MP
3. Generic UVC USB Cameras
 - a. Large variety of form factors, specifications and price points
 - b. Connects via USB and UVC Compliance allows it to work with the Raspberry PI 4B without the need for additional drivers

Once we have identified the appropriate visual marker and Image Sensor(s), we can move on to the next phase which is Identification/Data Reading and tracking. To do this, we can make use of the following software:

1. ZBar
 - a. ZBar is an open-source software that enables the reading and processing of data from QR Codes and Barcodes. This can help us in identifying the station the robot is at.
2. OpenCV
 - a. OpenCV is a powerful open source library that enables Computer Vision. It is able to read data from QR Codes, and identify and track ArUco markers.

2.3 Payload Delivery Mechanism

A compact and reliable Payload Delivery Mechanism is required to autonomously deliver the payload (ping pong balls) from the robot into a target receptacle. The system must provide consistent launch velocity, accurate trajectory and sequential firing (if required) while operating within the size, weight and power constraints of the TurtleBot platform. It must function reliably under varying docking distances and alignment conditions to ensure successful payload delivery to both static and moving targets. We will split this into two stages:

Stage 1: Payload Storage

1. Bowl
 - a. Depending on the height of the target receptacle, we can place a bowl in between the Raspberry Pi (Layer 3) and the LiDAR (Layer 4) or the OpenCL (Layer 2) to store the Balls before launch. Payloads are fed into the launch mechanism through gravity. This enables us to develop a more compact robot as the bowl can be confined to the width dimensions of the robot while still being able to hold the required number of balls needed for the mission.
2. Wrap Around Shaft (See Appendix A.1)
 - a. In this design, a Payload shaft that wraps around the body is used for storage. The Payload
 - b. Shaft is angled downwards, ensuring that the payloads are able to travel down the spiral and eventually feed into the delivery mechanism.

Stage 2: Delivery Mechanism

1. Flywheel
 - a. In this design, we can use two high-speed flywheels to impart velocity to the payload and a separate feeder (pusher) motor to advance the payload into the flywheel channel for sequential firing. By regulating the speed of the flywheels, we can adjust the launch speed and trajectory of the payload. The flywheels can be driven by either one or two motors.
2. Spring Loaded
 - a. In this design, a servo can be used to pull back on an elastic medium (spring, elastic band, etc) connected to a firing pin, creating tension. To deliver the payload, the servo simply releases the elastic medium, causing the energy stored in the elastic medium to transfer to the firing pin which pushes the payload out of the launch mechanism. The trajectory of the payload can be adjusted by changing the distance the elastic medium is pulled and the medium's elasticity

Conceptual Design (Step 3)

Based on our research, we have come up with two potential designs for our robot:

Concept 1:

- Navigation
 - Path Planning: Breadth First Search Frontiers Exploration
 - Path Execution: Regulated Pure Pursuit
- Visual Marker Identification and Tracking
 - Visual Marker: ArUco Marker
 - Image Sensor: Raspberry Pi Camera Module 2 + 2 Logitech CS270 USB Webcam
 - Identification and Tracking Software: OpenCV
- Payload Delivery System
 - Payload Storage: Wrap Around Shaft
 - Payload Delivery: Flywheel

Concept 2:

- Navigation
 - Path Planning: Wall Follower
 - Path Execution: Dynamic Window Approach
- Visual Marker Identification and Tracking
 - Visual Marker: QR Code
 - Image Sensor: Raspberry Pi Camera Module 2 + 2 Generic UVC USB Cameras
 - Identification and Tracking Software: OpenCV
- Payload Delivery System
 - Payload Storage: Bowl
 - Payload Delivery: Spring Loaded

Concept Evaluation (Step 4: BOGAT)**4.1 Tradeoffs**

PATH PLANNING		
Method	Advantage	Disadvantage
Wall Follower	Simple navigation method that is easy to implement	Fails if there are missing wall panels or there are loops in the maze. Especially if they are perimeter walls
	Predictable Behaviour	Not the fastest Navigation Method
Frontiers Exploration	Ensures Map Completeness as the Robot will attempt to explore all frontiers available in the maze	Might mistake as missing perimeter walls in the maze as possible frontiers
Best Search First (A*)	Ensures the safety of the robot during operation by taking into account the associated 'costs' in each step	Computationally & Resource Heavy
		Requires a Goal Point to be set first
PATH EXECUTION		
Method	Advantage	Disadvantage
Regulated Pure Pursuit	Adaptive Nature Allows for better stability of the Robot during movement, especially when executing turns	May have issues in high to extreme tight turns, potentially leading to collisions with obstacles or deviations from path
	Easy to implement	
Dynamic Window Approach	Ensures that the robot does not exceed its physical limits when encountering obstacles	May not be able to account for dynamic obstacles
Visual Marker Type		
Method	Advantage	Disadvantage
QR Code	Able to encode large amounts of data	Mediocre at Motion tracking due to limited readability angles and distances
ArUco Marker	Excels at Motion tracking	Can only store its identification number
Image Sensor		
Raspberry Pi Camera Module 2	Integrates natively into the Raspberry Pi ecosystem	CSI Ribbon cable is quite fragile
	Compact form factor due to the lack of casing	Requires Calibration to ensure good image output
Logitech CS270	Plug and Play Convenience	Built in casing is bulky and modifications required to slim it down may damage the module

	Works without the need for additional drivers	Video quality may not be the best, especially at its price point
	Durable and Reliable	Limited FOV
Generic UVC USB Cameras	Comes in a variety of form factors and specifications which we can choose to suit our needs	Quality and Reliability of Product cannot be guaranteed due to the Generic nature
	Large Price Range which enables us to find options that are within budget	Some options comes with a casing that may need to be modified to fit within the size restrictions of our robot
Identification and Tracking		
Method	Advantage	Disadvantage
ZBar	Lightweight Library that is easy to implement	Can only read QR Codes and barcodes
		Cannot do motion tracking
OpenCV	Able to many types of visual markers including QR Codes and ArUco Markers	May be computationally/resource heavy
	Able to do motion/object tracking	
Payload Storage		
Method	Advantage	Disadvantage
Bowl	Large Amount of ping pong balls can be stored while still keeping to a compact form factor	Higher Probability of balls getting jammed within the bowl and not falling into the feeder mechanism
	Easy to build and implement	Added height may result in a higher center of gravity which may cause instability issues during tight turns
Wrap Around Shaft	Enables Controlled and Continuous feeding of payload into the delivery mechanism	Mechanically Complex and Requires precise fabrication and is harder to maintain
	Reduces Jamming Compared to Static Storage methods	
	Maintains compact vertical profile	
Payload Delivery		
Method	Advantage	Disadvantage
Spring Loaded	Simple and Lightweight Design	Force degrades over time
	Consistent Launch Force when calibrated	Limited firing range
	Low Power Consumption (energy stored mechanically)	Requires manual or motorized reloading
Flywheel	High firing rate and continuous launch capability	Higher power consumption
	Adjustable launch speed by varying motor RPB	Requires precise alignment and speed control
	Reliable and repeatable projectile velocity	Increased risk of ball slippage at higher or inconsistent speeds

4.3 Selected Concept

Navigation

Primary System: Frontiers Exploration with Regulated Pure Pursuit

Back Up System: Wall Follower

Reason:

Frontiers Exploration allows us to explore the map fully in a short amount of time as there will always be a new Frontier that has not been explored yet until map completion. By using Regulated Pure Pursuit to execute planned routes, it ensures that the robot can move around at higher speeds while still being able to guarantee its safety. With a fallback to Wall Following, the hybrid approach to navigation allows us to have a robust and reliable navigation system.

Visual Marker Identification

Marker Type: ArUco Markers

Camera Layout: Triple Camera Set Up

Camera Type: Raspberry Pi Camera Module 2 (RPCM2) + 2 Generic UVC USB Cameras (GUUC)

Detection Software: OpenCV

Reason:

ArUco Markers excels in motion tracking scenarios which is key in this mission context. Although they are not able to store as much information as their QR Code counterparts, their ID Number is more than sufficient enough to help distinguish them from each other.

A triple camera set-up which has cameras placed at the front, left and right sides of the robot allows us to have a better field of view to detect the Visual Markers. The mix of GUUCs as well as the RPCM2 allows us to have a more robust system as the GUUC allows us a degree of freedom to choose a sensor that fits our requirements and the RPCM2 guarantees smooth integration into the Turtlebot 3 system.

OpenCV is a powerful library that enables us to do complex Computer Visioning. It does the job of ZBar in both reading and identifying QR Codes (if needed) and also allows us to write scripts to identify and track ArUco Markers which is key in this mission.

Payload Delivery System

Storage Method: Wrap Around Shaft

Launcher Mechanism: Flywheel

Reason:

The Wrap Around Shaft design decreases the chances of payloads getting jammed within the holder. This, combined with the flywheel mechanism, allows for a smooth, reliable and consistent delivery. The flywheel mechanism also makes it easier for us to calibrate as launch speed and trajectory is adjusted by the angular velocity of the flywheels.

To achieve controlled payload release, a dedicated feeder motor is used to advance the projectile from the wrap-around shaft into the flywheel interface. This decouples payload storage from the launching action, ensuring that only a single projectile is fed at a time. The feeder motor allows precise timing and positioning of the payload before launch, while the flywheels handle energy transfer. This separation of feeding and launching improves consistency, simplifies calibration, and enhances overall reliability on the TurtleBot platform.

Preliminary Design (Step 5)

5.1 System Architecture:

Sensors → Perception & Localisation → Planning → Control → Actuation

- LIDAR provides mapping and pose estimation
- Camera detects visual markers for station identification and docking
- Frontier exploration generates waypoints for full maze coverage
- Regulated Pure Pursuit executes safe path tracking
- Launcher subsystem performs sequential payload delivery

A mission state machine governs operation: initialise → explore/map → navigate to station → align → deliver payload → proceed to next objective

5.2 Hardware Architecture:

The system is built on the Turtlebot 3 platform with integrated sensing, computation and delivery subsystems

Sensors

- LiDAR for mapping and obstacle detection
- Camera for marker detection and alignment
- IMU and wheel encoders for pose estimation

Payload Subsystem

- Wrap-around shaft enables controlled gravity-fed storage
- Dual flywheel launcher ensures consistent launch velocity
- Feeder mechanism enables sequential delivery

Key integration considerations:

- Low centre of gravity for stability
- Rigid mounting to minimise vibration
- Compact launcher placement to avoid LIDAR obstruction

5.3 Software Architecture:

A modular ROS2-based architecture will be used

Core nodes

- SLAM/localisation
- Exploration and path planning
- Navigation control
- Vision detection
- Launcher control

A supervisory state machine coordinates subsystem operation and ensures payload deployment only occurs after successful localisation and alignment

5.4 Subsystem Interfaces:

- Navigation outputs velocity commands to motor controllers
- Vision system provides marker pose for fine alignment
- Alignment confirmation triggers launcher sequence
- State machine provides supervisory control and safety override

Clear interface definition ensures reliable subsystem integration and traceability

5.5 Verification Plan and Risk Considerations

Verification will be conducted at the subsystem and system levels:

- Navigation accuracy and collision avoidance
- Marker detection and docking alignment reliability
- Launcher consistency and sequential delivery success rate
- Full mission timing and autonomy validation

Key risks include marker detection failure, payload jamming and localisation drift.

Mitigation measures include fallback search behaviours, controlled feed design and conservative navigation speeds.

5.6 Performance Measures

System performance will be evaluated using the following Measure of Performance (MOPs):

- **Mission Completion:** Full mission completed within 25 minutes
- **Mission Success Rate:** $\geq 90\%$ successful autonomous runs
- **Navigation:** Zero wall collisions and complete maze coverage
- **Marker Detection:** $\geq 95\%$ detection reliability
- **Target Tracking:** $\geq 90\%$ accuracy for moving station
- **Payload Delivery:** $\geq 95\%$ successful deliveries into receptacle
- **System Reliability:** No system crash during mission execution

These MOPs will be used as Technical Performance Measures (TPMs) during testing to ensure mission readiness

5.7 Mission Success Rate

We will define the Mission to be successful if the following criterias are met:

1. Robot Successfully Identifies and Navigates to both Station A and B
2. 5 out of 6 payloads are successfully delivered into the target receptacle. This includes all 3 payload delivered at Station A and 2 out of 3 for Station B.

All of these criteria must be done within the stipulated timeframe of 25 minutes.

5.8 Failure Modes

1. Back Up Navigation Algorithm
 - a. In the event the primary method of Frontiers Exploration Fails due to extreme changes in the robot's operating environment, we will implement an automatic switch to a wall following algorithm for navigation until the situation stabilises. This ensures that the robot will still have access to a reliable navigation method, although it would increase the time needed to fully explore the maze.
2. Back Up Cameras
 - a. By using a triple camera set-up, it guarantees that the robot will still be able to detect visual markers in the event of a single or dual camera failure.

Conclusion

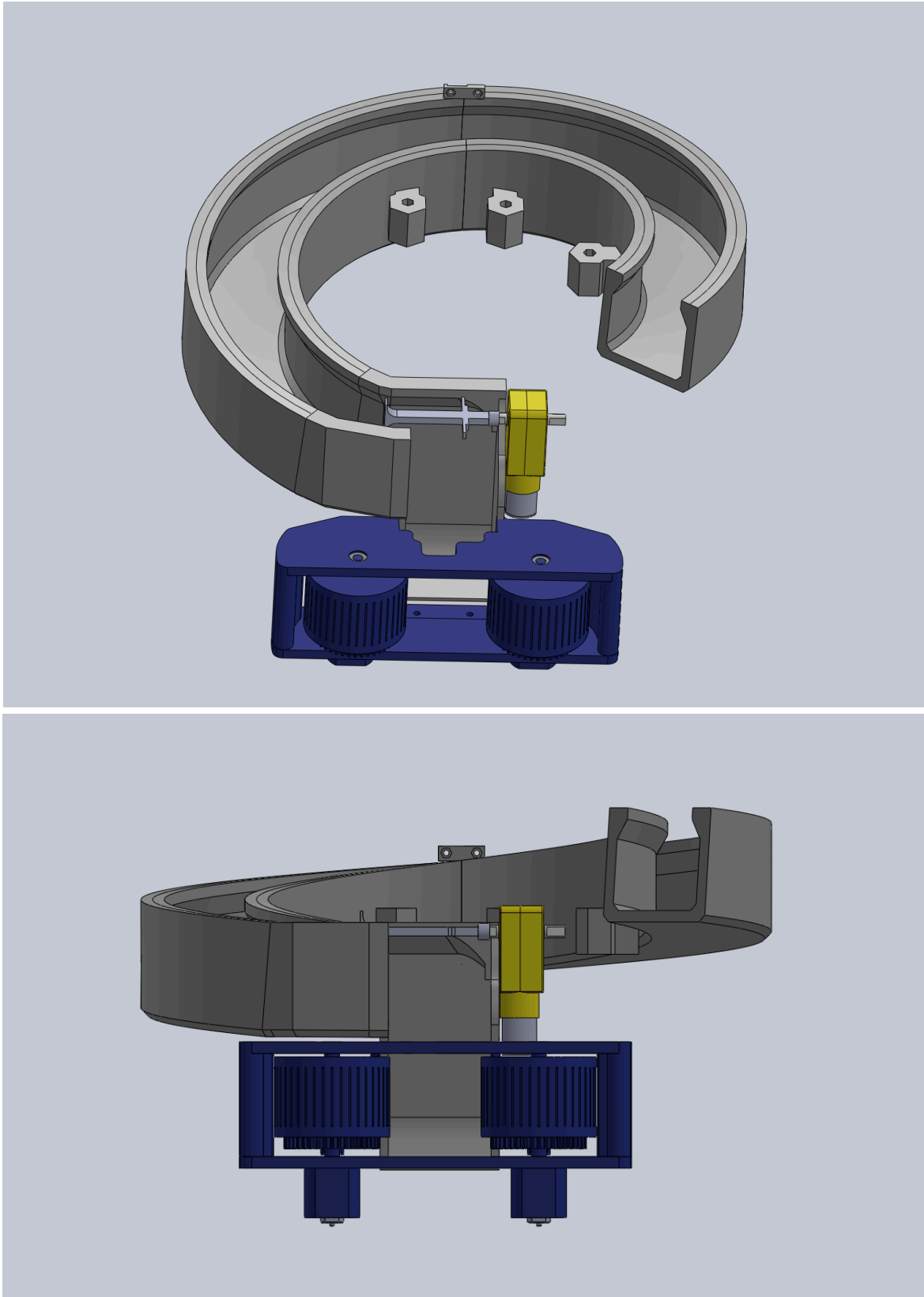
The proposed system design ensures that it is capable of performing complex warehouse delivery tasks under defined operational constraints. The primary and back-up navigation approach outlined ensures that the robot has a safe, robust and reliable method of navigation through the unknown environment, and the Visual Marker Detection system, coupled with the Flywheel delivery mechanism ensures that the payloads are delivered in a reliable, safe and timely manner.

Moving forward, we will focus on refining our ideas, finalizing the CAD Model, and conducting system level tests ahead of the Preliminary Design Review (PDR) to ensure the effectiveness and reliability of the ideas proposed. We will also finalise the budget proposal and bill of materials for the PDR as well.

Appendix

A.1 CAD Concepts:

Wrap Around Shaft Concept:



A.2 References:

A.2.1 Mission Documentation:

Chew, N. (2025). CDE2310-AY25/DOCS/Mission Readme.md at Main · Edic-NUS/CDE2310-AY25. CDE2310. <https://github.com/edic-nus/CDE2310-AY25/blob/main/docs/Mission%20Readme.md>

A.2.2 Navigation:

ROBOTIS e-Manual. (n.d.). ROBOTIS E-Manual. Retrieved 18 February 2026, from <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>

Nav2—Nav2 1.0.0 documentation. (n.d.). Retrieved 18 February 2026, from <https://docs.nav2.org/>

Maze-solving algorithm. (2026). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Maze-solving_algorithm&oldid=1337762285

Tjiharjadi, S. (2020). *Performance Comparison Robot Path Finding Uses Flood Fill—Wall Follower Algorithm and Flood Fill—Pledge Algorithm* [Maranatha Christian University]. <https://pdfs.semanticscholar.org/29ff/3a714d952c1fe22b25c2b205f96c4d2c948a.pdf>

Yamauchi, B. (1998). Frontier-based exploration using multiple robots. *Proceedings of the Second International Conference on Autonomous Agents - AGENTS '98*, 47–53. <https://doi.org/10.1145/280765.280773>

Nakamura, K. (2024, May 15). *How to Make an Autonomous Mapping Robot Using SLAM* [Video recording]. <https://www.youtube.com/watch?v=xqjVTE7QvOg>

Topiwala, A., Innia, P., & Kathpal, A. (n.d.). Frontier Based Exploration for Autonomous Robot [University of Maryland]. Retrieved <https://arxiv.org/pdf/1806.03581>

Buriboev, A., Muminov, A., Oh, H., Lee, J. D., Kwon, Y., & Jeon, H. S. (2021). Internal and external frontier-based algorithm for autonomous mobile robot exploration in unknown environment. *Electronics Letters*, 57(24), 942–944. <https://doi.org/10.1049/ell2.12316>

Breadth first search or bfs for a graph. (2012, March 20). GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>

McKee, A. (2024, November 3). *Depth-First Search in Python: Traversing Graphs and Trees*. datacamp. <https://www.datacamp.com/tutorial/depth-first-search-in-python>

Best first search(Informed search). (2017, May 3). GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/best-first-search-informed-search/>

Regulated Pure Pursuit—Nav2 1.0.0 documentation. (n.d.). Retrieved 18 February 2026, from <https://docs.nav2.org/configuration/packages/configuring-regulated-pp.html>

Macenski, S., Singh, S., Martin, F., & Gines, J. (2023). *Regulated pure pursuit for robot path tracking*. <https://doi.org/10.48550/ARXIV.2305.20026>

Fox, D., Burgard, W., & Thrun, S. (1997). *The Dynamic Window Approach to Collision Avoidance* [University of Bonn, Carnegie Mellon University]. https://www.ri.cmu.edu/pub_files/pub1/fox_dieter_1997_1/fox_dieter_1997_1.pdf

Dynamic window approach. (2025). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Dynamic_window_approach&oldid=1305732122

Cao, Y., & Mohamad Nor, N. (2024). An improved dynamic window approach algorithm for dynamic obstacle avoidance in mobile robot formation. *Decision Analytics Journal*, 11, 100471. <https://doi.org/10.1016/j.dajour.2024.100471>

A2.2.3 Visual Marker Identification and Tracker:

Fologram—Tracking of mobile/movable qr-codes(Object-/motion tracking). (2024, March 14). McNeel Forum. <https://discourse.mcneel.com/t/fologram-tracking-of-mobile-movable-qr-codes-object-motion-tracking/177960>

Opencv: Detection of aruco markers. (n.d.). Retrieved 18 February 2026, from https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

Raspberry Pi Camera Module 2. (n.d.). Raspberry Pi. Retrieved from <https://www.raspberrypi.com/products/camera-module-v2/>

Logitech c270 hd webcam, 720p video with noise reducing mic. (n.d.). Logitech. Retrieved 18 February 2026, from <https://www.logitech.com/en-sg/shop/p/c270-hd-webcam>

Nayak, S. (2020, March 21). *Augmented reality using aruco markers in opencv (C++ / python) | learnopencv #*. <https://learnopencv.com/augmented-reality-using-aruco-markers-in-opencv-c-python/>

Bovbel, P. (n.d.). *Zbar_ros* [C++]. ROS.org. Retrieved 18 February 2026, from https://wiki.ros.org/zbar_ros

Opencv—Open computer vision library. (n.d.). OpenCV. Retrieved 18 February 2026, from <https://opencv.org/>

A2.2.4 Payload Delivery Mechanism:

Butler, N. (2021, April 18). Automatic ping-pong launcher. Medium. <https://nollaigengineering.medium.com/automatic-ping-pong-launcher-c77dc140d33b>

Jacques, T. (2024, July 26). *HIGH-SPEED Ping-Pong Ball Shooter XRP*. Printables.com. Retrieved February 18, 2026, from <https://www.printables.com/model/951454-high-speed-ping-pong-ball-shooter-xrp>